

1 La bibliothèque d'entrée/sortie asm_io

Exercice 1. En observant le programme `Add.asm`, expliquer

1. ce que fait la ligne 18

On affecte la valeur de `eax` dans l'adresse `[input1]`. Cad l'entier saisi au clavier est enregistré dans `input1`.

Cette ligne prend la valeur saisie par l'utilisateur (qui a été lue par `read_int` juste avant et placée dans le registre `eax`), puis la stocke à l'adresse mémoire réservée pour `input1`. Autrement dit, le premier nombre tapé au clavier est enregistré dans la variable `input1`.

Registre : zone mémoire très rapide du CPU.

`[input1]` : zone mémoire réservée pour garder la valeur.

2. ce que fait la ligne 24

On additionne `eax` (contient la val à l'adresse `[input1]`) et la valeur à l'adresse `[input2]`.

Elle ajoute à la valeur présente dans le registre `eax` celle qui est stockée à l'adresse `input2`. À ce moment : `eax` contient déjà la valeur de `input1` (rapportée ligne 23 avec `mov eax, [input1]`). `add eax, [input2]` fait donc : `eax=input1+input2` Le résultat (la somme) est dans `eax`.

3. ce que fait la ligne 29

On appelle `print_int` qui permet d'afficher la valeur dans `eax` (le résultat de la somme).

Elle affiche sur la sortie standard la valeur entière contenue dans le registre `eax`. Ici, cette valeur est la somme des deux nombres saisis par l'utilisateur, qu'on a copiée de `ebx` vers `eax` juste avant (`mov eax, ebx` en ligne 28).

4. ce que font les lignes 31, 32 et 33.

Permet de terminer le programme (31. 0 signifie que tout s'est bien passé = code de retour du script).

Ligne 31 : met la valeur 0 dans `ebx`, ce qui sera le code de retour du programme.

Ligne 32 : met la valeur 1 dans `eax`, qui est le numéro de l'appel système `exit` en Linux/x86 (pour l'instruction d'appel système). Ligne 33 : `int 0x80` provoque l'interruption système pour exécuter l'appel système décrit par `eax`, c'est-à-dire terminer le programme. Résumé : ensemble, ces trois lignes permettent de terminer correctement le programme et retourner la valeur 0 au système.

2 Les sauts inconditionnels/conditionnels

Exercice 2. Considérons la suite d'instructions

```
mov eax, 0xFFFFFFFF
cmp eax, 0
jg aff_1
mov eax, 0
call print_int
aff_1 :
mov eax, 1
call print_int
```

1. Expliquer ce qu'elles affichent en précisant ce que fait chaque étape de l'exécution.
2. Reprendre la question précédente en considérant la suite d'instructions obtenue en remplaçant jg par ja en ligne 3.

1.

mov eax, 0xFFFFFFFF	Dans le registre eax, on met la val 0xFFFFFFFF (vaut -1)
cmp eax, 0	Comparaison entre eax et 0
jg aff_1	Saute à aff_1 si eax > 0 (en signé), sinon continue à l'instruction suivante
mov eax, 0	Dans le registre eax, on met la val 0
call print_int	Appeler print_int (affiche sur la sortie standard l'entier signé contenu dans eax soit l'entier 0 positif ?)
aff_1 :	Etiquette aff_1 (Si eax = 0 on y saute direct) mais sinon fin du bloc dans tous les cas exécuté
mov eax, 1	Dans le registre eax, on met la val 1
call print_int	Appeler print_int (affiche sur la sortie standard l'entier signé contenu dans eax soit l'entier 1)

Donc la sortie standard affiche 0, puis j'affiche 1. Parce que le saut n'a PAS lieu (eax = -1, donc pas plus grand que 0 en signé).

2.

mov eax, 0xFFFFFFFF	Dans le registre eax, on met la val 0xFFFFFFFF
cmp eax, 0	Comparaison entre eax et 0
ja aff_1	Il saute si CF == 0 ET ZF == 0 (SANS retenue & PAS égal). Avec cmp eax, 0, et eax = 0xFFFFFFFF (= 4294967295 non signé), alors saut effectué car 4294967295 > 0. Donc on saute immédiatement à aff_1 et on affiche 1 (pas 0). * CF , « Carry Flag », vaut 1 si l'instruction produit une retenue

	de sortie et 0 sinon; * ZF , « Zero Flag », vaut 1 si l'instruction produit un résultat nul et 0 sinon ;
mov eax, 0	Dans le registre eax, on met la val 0
call print_int	Appeler print_int (affiche sur la sortie standard l'entier signé contenu dans eax soit l'entier 0 positif ?)
aff_1 :	Etiquette aff_1 (Si eax != 0 et pas de retenue)
mov eax, 1	Dans le registre eax, on met la val 1
call print_int	Appeler print_int (affiche sur la sortie standard l'entier signé contenu dans eax soit l'entier 1)

Donc la sortie standard affiche 1.

jb : saute si résultat de la comparaison (signée) est > 0. Ici, -1 n'est PAS > 0 donc pas de saut.

ja : saute si (unsigned) eax > 0. Ici, 0xFFFFFFFF = 4294967295 > 0 donc saut.

La différence se situe dans la prise en compte du signe : même valeur binaire, deux interprétations possibles.

Exercice 3. Écrire un programme E3.asm qui lit deux entiers au clavier et affiche le maximum.

```
camelia.antoine@pccoplbl140-05:~/Documents/Archi/TP3$ nano E3.asm
camelia.antoine@pccoplbl140-05:~/Documents/Archi/TP3$ nano E3.asm
camelia.antoine@pccoplbl140-05:~/Documents/Archi/TP3$ nano Compile.sh
camelia.antoine@pccoplbl140-05:~/Documents/Archi/TP3$ chmod +x Compile.sh
camelia.antoine@pccoplbl140-05:~/Documents/Archi/TP3$ ./Compile.sh E3
```

```

Fichier Éditeur Affichage Rechercher Terminal Aide
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ nano E3.asm
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ nano E3.asm
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ nano Compile.sh
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ chmod +x Compile.sh
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ ./Compile.sh E3
E3.asm:5: error: unable to open include file 'asm_io.inc': No such file or directory
ld : ne peut pas trouver E3.o : Aucun fichier ou dossier de ce nom
ld : ne peut pas trouver asm_io.o : Aucun fichier ou dossier de ce nom
chmod: impossible d'accéder à 'E3': Aucun fichier ou dossier de ce nom
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ ./Compile.sh E3
ld : ne peut pas trouver asm_io.o : Aucun fichier ou dossier de ce nom
chmod: impossible d'accéder à 'E3': Aucun fichier ou dossier de ce nom
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ nano Compile.sh
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ ./Compile.sh asm_io
ld : asm_io.o : dans la fonction « read_int » :
asm_io.asm:(.text+0x0) : définitions multiples de « read_int » ; asm_io.o:asm_io.asm:(.text+0x0) : défini pour la première fois ici
ld : asm_io.o : dans la fonction « print_string » :
asm_io.asm:(.text+0x19d) : définitions multiples de « print_string » ; asm_io.o:asm_io.asm:(.text+0x19d) : défini pour la première fois ici
ld : asm_io.o : dans la fonction « print_espace » :
asm_io.asm:(.text+0x1ce) : définitions multiples de « print_espace » ; asm_io.o:asm_io.asm:(.text+0x1ce) : défini pour la première fois ici
ld : asm_io.o : dans la fonction « print_nl » :
asm_io.asm:(.text+0x1db) : définitions multiples de « print_nl » ; asm_io.o:asm_io.asm:(.text+0x1db) : défini pour la première fois ici
ld : asm_io.o : dans la fonction « print_int » :
asm_io.asm:(.text+0x1fa) : définitions multiples de « print_int » ; asm_io.o:asm_io.asm:(.text+0x1fa) : défini pour la première fois ici
ld : avertissement : le symbole d'entrée main est introuvable ; utilise par défaut 08049000
chmod: impossible d'accéder à 'asm_io': Aucun fichier ou dossier de ce nom
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ nasm -f elf32 asm_io.asm
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ ./Compile.sh E3
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ ./E3
Entrer un nombre : 4
Un autre nombre : 6
Le maximum est : camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ ./Compile.nano E3.asm
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ ./Compile.sh E3
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ ./E3
Entrer un nombre : 5
Un autre nombre : 10
Le maximum est : 10camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ ./Compilnano E3.asm
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ ./Compile.sh E3
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$ ./E3
Entrer un nombre : 2
Un autre nombre : 5
Le maximum est : 5
camelia.antoine@pccopl140-05:~/Documents/Archi/TP3$

```

- ; Auteurs : Camelia
- ; Date de creation : 10/10/25
- ; Objectif : lire 2 entiers et afficher le maximum.

```
%include "asm_io.inc"
.....
,,,,,,,,,,,,,,
..... Section de donnees .....
,,,,,,,,,,,,,,
SECTION .data
prompt1 : db "Entrer un nombre : ", 0
prompt2 : db "Un autre nombre : ", 0
outmsg1 : db "Le maximum est : ", 0
.....
,,,,,,,,,,,,,,
..... Section de reservation .....
```

```
input1 : resd 1 ;je reserve 4 octet pour l'input1
input2 : resd 1 ;je reserve 4 octet pour l'input2
```

SECTION .text

main:

```

mov eax, prompt1
call print_string ; Affichage de prompt1.
call read_int ; Lecture d'un entier.
mov [input1], eax
mov eax, prompt2
call print_string ; Affichage de prompt2.
call read_int ; Lecture d'un entier.
mov [input2], eax

cmp eax, [input1]
jl else
    mov eax, outmsg1 ;input2 > input1
    call print_string
    mov eax, [input2]
    call print_int
    jmp end_if

else: ;input2 < input1
    mov eax, outmsg1
    call print_string
    mov eax, [input1]
    call print_int

end_if ; Fin de l'execution.
call print_nl
mov ebx, 0
mov eax, 1
int 0x80

```

```

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
SECTION .data
prompt1 : db "Entrer un nombre : ", 0
prompt2 : db "Un autre nombre : ", 0
outmsg1 : db "Le maximum est : ", 0

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
; Section de reservation ;
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
SECTION .bss
input1 : resd 1 ;je reserve 4 octet pour l'input1
input2 : resd 1 ;je reserve 4 octet pour l'input2

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
; Section de code ;
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
SECTION .text

global main
main:
    mov eax, prompt1
    call print_string    ; Affichage de prompt1.
    call read_int        ; Lecture d'un entier.
    mov [input1], eax
    mov eax, prompt2
    call print_string    ; Affichage de prompt2.
    call read_int        ; Lecture d'un entier.
    mov [input2], eax

    cmp eax, [input1]
    jl else
        mov eax, outmsg1    ;input2 > input1
        call print_string
        mov eax, [input2]
        call print_int
        jmp end_if

    else:    ;input2 < input1
        mov eax, outmsg1
        call print_string
        mov eax, [input1]
        call print_int

    end_if: ; Fin de l'execution.
    call print_nl
    mov ebx, 0
    mov eax, 1
    int 0x80

```

```
%include "asm_io.inc"
```

```
SECTION .data
```

```
prompt1 : db "Premier entier : ", 0
```

```
prompt2 : db "Deuxieme entier : ", 0
```

```
outmsg : db "Le maximum est : ", 0
```

```
SECTION .bss
```

```
input1 : resd 1
```

```
input2 : resd 1
```

```
SECTION .text
```

```
global main
```

```
main :
```

```
    ; Afficher prompt1 et lire input1
```

```
    mov eax, prompt1
```

```
    call print_string
```

```
    call read_int
```

```
    mov [input1], eax
```

```
    ; Afficher prompt2 et lire input2
```

```
    mov eax, prompt2
```

```
    call print_string
```

```
    call read_int
```

```
    mov [input2], eax
```

```
    ; Comparaison
```

```
    mov eax, [input1]    ; Charger input1 dans eax
```

```
    cmp eax, [input2]    ; Comparer avec input2
```

```
    jge input1_is_maximum ; Si input1 >= input2, on saute à l'affichage de input1
```

```
    mov eax, [input2]    ; Sinon, on met input2 dans eax
```

```
    jmp afficher_max
```

```
input1_is_maximum:
```

```
    mov eax, [input1]
```

```
afficher_max:
```

```
    mov ebx, eax    ; Sauver le max
```

```
    mov eax, outmsg
```

```
    call print_string
```

```
    mov eax, ebx
```

```
    call print_int
```

```
    call print_nl
```

```
    mov ebx, 0
```

```
    mov eax, 1
```

```
    int 0x80
```

Exercice 4. Donner les valeurs de **eax** et de **edx** après les instructions

```
mov edx, 0x00000000 => 0
mov eax, 0x000005DE => 1502
mov ebx, 15
div ebx => 1502 / 15 = 100 + 2 => 100 représente le quotient et 2 le reste. OU
1502-(100×15)=2
```

Donc **edx** vaut 0x00000002 (2) et **eax** vaut 0x00000064(100).

Exercice 5.

1. Écrire un programme **E5.asm** qui lit deux nombres **a** et **b** au clavier et qui affiche « **Oui** » si **a** est divisible par **b** et « **Non** » sinon. Si la réponse est négative, le reste de la division sera affiché.

Structure générale à compléter

- Lire deux entiers au clavier : **a** et **b**
- Tester si **a** est divisible par **b**
- Si oui : afficher "Oui"
- Si non : afficher "Non" et le reste

Pseudocode logique

- Demander **a**, lire **a**
- Demander **b**, lire **b**
- Division entière : **a / b**
- Si le reste est 0, afficher "Oui", sinon "Non" et afficher le reste.

```
%include "asm_io.inc"
```

```
SECTION .data
```

```
msg_a    db "Entrez a : ", 0
msg_b    db "Entrez b : ", 0
oui       db "Oui", 0
non       db "Non", 0
restemsg db "Le reste est : ", 0
```

```
SECTION .bss
```

```
input_a  resd 1
```

```
input_b  resd 1
```

```
SECTION .text
```

```
global main
```

```
main:
```

```
    mov eax, msg_a
    call print_string
```



```

call read_int
mov [input_a], eax

mov eax, msg_b
call print_string
call read_int
mov [input_b], eax

mov eax, [input_a]
mov ebx, [input_b]
mov edx, 0          ; Pour DIV : EDX doit être 0 (pas de bits poids fort)
div ebx             ; EAX = quotient, EDX = reste

cmp edx, 0          ; Teste si le reste est 0
je affiche_oui
mov eax, non
call print_string
call print_espace
mov eax, restemsg
call print_string
mov eax, edx         ; Affiche le reste
call print_int
call print_nl
jmp fin

affiche_oui:
mov eax, oui
call print_string
call print_nl

fin:
mov ebx, 0
mov eax, 1
int 0x80

```

2. Expliquer ce qu'il se passe lorsque ce programme prend **a := -1** et **b := 17** en entrée.

Dans la version avec div, attention ! Toutes les opérations sont faites en non signé. Si tu mets a = -1, cela vaut 0xFFFFFFFF (en non signé, très grand nombre, en signé, c'est -1). Cela crée une division non signée, donc bizarre si on rentre des valeurs négatives : La division peut alors provoquer une erreur ou donner un résultat hors du sens attendu. Le reste obtenu sera 0xFFFFFFFF % 17, qui est différent de ce qu'on attend en mathématiques pour -1 % 17 (normalement, le reste signé est -1 ou 16 selon convention).

3. Améliorer le programme précédent en un programme **E+.asm** qui fonctionne correctement avec les entiers signés en utilisant **idiv** à la place de **div**. Pour cela, on

peut utiliser l'instruction **cdq** (Convert Double word to Quad word) pour étendre le signe de **eax** dans **edx:eax**.

Pour gérer les entiers signés :

- On utilise **idiv** au lieu de **div**
- On doit étendre le signe de **EAX** dans **EDX** avec **cdq** avant de diviser

```
mov eax, [input_a]
mov ebx, [input_b]
cdq          ; EDX reçoit le même signe que EAX
idiv ebx     ; Quotient dans EAX, reste dans EDX
```

Si tu entres $a = -1$ et $b = 17$,

- **idiv** donne Quotient = 0, Reste = -1
- Donc le test **cmp edx, 0** sera faux (re. non nul)
- On affiche "Non", et le reste qui vaut -1

Exercice 6.

Écrire un programme **E6.asm** qui lit un entier strictement positif au clavier et affiche tous les entiers qui divisent ce nombre. Par exemple si le nombre lu est 60, le programme affichera

1 2 3 4 5 6 10 12 15 20 30 60

```
%include "asm_io.inc"
```

```
SECTION .data
```

```
prompt db "Veuillez entrer un entier strictement positif : ", 0
```

```
SECTION .bss
```

```
n      resd 1      ; stocke l'entier saisi
```

```
SECTION .text
```

```
global main
```

```
main:
```

```
    ; Lire n
```

```
    mov eax, prompt
```

```
    call print_string
```

```
    call read_int      ; EAX = n
```

```
    mov [n], eax
```

```
    ; i va aller de 1 à n
```

```
    mov ecx, [n]      ; ECX = n → sera notre compteur pour loop
```

```
    mov ebx, 1        ; EBX = i = 1 (diviseur courant)
```

```
boucle:
```

```
    ; tester si i (EBX) divise n
```

```
    mov eax, [n]      ; EAX = n
```

```

mov edx, 0      ; EDX = 0 pour div unsigned
div ebx         ; n / i → quotient EAX, reste EDX

```

```

cmp edx, 0
jne pas_diviseur ; si reste ≠ 0, i n'est pas diviseur

```

```

; ici, i = EBX est un diviseur → on l'affiche
mov eax, ebx
call print_int
call print_espace

```

```

pas_diviseur:
inc ebx      ; i++ (diviseur suivant)
loop boucle  ; ECX-- puis si ECX != 0, retourne à boucle

; fin de ligne
call print_nl

; sortie propre
mov ebx, 0
mov eax, 1
int 0x80

```

1. Lire le nombre à traiter

- Affiche un message pour demander à l'utilisateur de saisir l'entier positif.
- Utilise `read_int` pour lire ce nombre et le stocker (dans `input` par exemple).

2. Préparer une boucle de 1 à n

- Tu vas utiliser un compteur (appelons-le `i`) qui démarre à 1.
- Pour chaque valeur de `i` de 1 à le nombre saisi, tu vas vérifier si `i` divise le nombre SANS reste.

3. Vérifier la divisibilité pour chaque `i`

- Pour cela : charge la valeur à tester dans `EAX`, place 0 dans `EDX` (pour la division), utilise `div` pour faire la division entière par `i`.
- La structure générale d'une itération devient :
 1. Charger le nombre saisi dans `EAX`.
 2. Placer 0 dans `EDX`.
 3. Charger le diviseur actuel (`i`) dans un registre (par ex : `EBX`).
 4. Faire `div ebx`.
 5. Si `EDX` (reste) est 0, alors `i` est un diviseur, donc on affiche `i`.

4. Incrémenter `i` et répéter jusqu'à n

```
%include "asm_io.inc"
```

```
SECTION .data
```

```
prompt    db "Veuillez entrer un entier strictement positif : ", 0
```

```
SECTION .bss
```

```
n         resd 1
```

```
i         resd 1
```

```
SECTION .text
```

```
global main
```

```
main:
```

```
    mov eax, prompt
```

```
    call print_string
```

```
    call read_int      ; Lire l'entier
```

```
    mov [n], eax
```

```
    mov dword [i], 1    ; i = 1
```

```
boucle:
```

```
    mov eax, [n]        ; EAX = n
```

```
    mov ebx, [i]        ; EBX = i
```

```
    mov edx, 0          ; EDX = 0 pour division entière
```

```
    div ebx             ; EAX=quotient, EDX=reste
```

```
    cmp edx, 0
```

```
    jne suite          ; Si reste ≠ 0 on saute
```

```
    mov eax, [i]        ; Si diviseur, affiche i
```

```
    call print_int
```

```
    call print_espace
```

```
suite:
```

```
    inc dword [i]       ; i++
```

```
    mov eax, [i]
```

```
    cmp eax, [n]
```

```
    jbe boucle          ; Tant que i ≤ n
```

```
    call print_nl       ; Fin de ligne
```

```
    mov ebx, 0
```

```
    mov eax, 1
```

```
    int 0x80            ; Terminer programme
```

Exercice 7.

Écrire un programme E7.asm qui lit un entier et affiche sa décomposition binaire sur 32 bits. Par exemple, si le nombre lu est 10, le programme affichera :

00000000000000000000000000001010

On pourra utiliser la commande `shl reg, 1` qui décale les bits du registre `reg` d'un pas vers la gauche et qui récupère le bit qui est sorti dans le drapeau de retenue `CF`. Il est possible d'utiliser les sauts conditionnels `jc` et `jnc` qui sautent si la retenue est à 1 ou 0 respectivement.

Remarque 3. Le second opérande de l'instruction `shl` peut être le registre `cl` et uniquement ce registre. Pour cela, on écrit `shl reg, cl`.

```
%include "asm_io.inc"
```

```
SECTION .data
```

```
prompt    db "Entrez un entier : ",0
```

```
zero_char db "0",0
```

```
one_char  db "1",0
```

```
SECTION .bss
```

```
nombre    resd 1
```

```
SECTION .text
```

```
global main
```

```
main:
```

```
    ; Demande le nombre à l'utilisateur
```

```
    mov eax, prompt
```

```
    call print_string
```

```
    call read_int
```

```
    mov [nombre], eax
```

```
    ; Charger le nombre pour affichage
```

```
    mov ecx, 32      ; Compteur de bits (32)
```

```
    mov eax, [nombre] ; EAX = nombre
```

```
aff_bit:
```

```
    shl eax, 1      ; Décale d'un bit à gauche, bit de poids fort -> CF
```

```
    jc print1       ; Si CF = 1 -> afficher 1
```

```
    mov ebx, zero_char
```

```
    call print_string
```

```
    jmp suite
```

```
print1:
```

```
    mov ebx, one_char
```

```
    call print_string
```

suite:

```
    loop aff_bit      ; ECX--
    call print_nl

    mov ebx, 0
    mov eax, 1
    int 0x80          ; Quitter le programme
```

Explications

- Lecture : on stocke le nombre à afficher en binaire dans [nombre]
- Boucle : ecx compte 32 passages (un par bit)
- À chaque itération, shl eax, 1 décale le registre à gauche : le bit de poids fort part dans le CF (Carry Flag)
- Si CF vaut 1 : on affiche "1" (avec print1)
- Sinon on affiche "0"
- À la fin de la boucle, on affiche un saut de ligne (print_nl)

En assembleur, tu n'as pas besoin de "calculer" énormément : chaque nombre que tu entres (par exemple : 10) est interprété par le processeur et stocké dans un registre (comme eax) en binaire automatiquement.

Ce qui reste à faire, c'est lire chaque bit du nombre et l'afficher séquentiellement :

- Le registre, par exemple EAX, stocke toujours la valeur en binaire en interne, même si tu l'as entrée en décimal.
- Le code que tu écris n'a pas à convertir explicitement du décimal vers le binaire au sens arithmétique : tu vas juste afficher la représentation de chaque bit, de la position 31 à 0.

Exemple pratique :

- Tu rentres : 10
- En mémoire : EAX stocke 00000000000000000000000000001010 (binaire de 10)
- La boucle shl eax, 1 va, pour chaque des 32 bits, te donner à chaque tour le bit le plus à gauche pour que tu puisses l'afficher.

Pour bien comprendre :

- Quand tu entres un nombre décimal, ton programme de lecture le convertit déjà en binaire pour le stocker.
- Ce qui te reste à faire, c'est d'interroger et d'afficher chaque bit du registre : affichage, pas conversion.

Exercice 8.

Écrire un programme E8.asm qui demande un entier et affiche le nombre de bits de cet entier qui valent 1. Par exemple si l'entier vaut 0x0000F00F, le programme renverra 8.

Astuce 1. L'idée est dans un premier temps d'utiliser l'instruction shr, analogue à l'instruction shl reg, 1, mais qui décale cette fois-ci les bits du registre d'un pas vers la droite.

Plan du programme

1. Lire l'entier au clavier, le stocker (par exemple dans [n])
2. Initialiser un compteur de bits à 0 (compte1)
3. Boucler 32 fois :
 - Décaler à droite (shr), le bit de poids faible sort dans CF
 - Si le bit sorti vaut 1 (CF=1), incrémenter le compteur
4. Afficher le résultat

```
%include "asm_io.inc"
```

SECTION .data

```
prompt    db "Entrez un entier : ",0
msg_result db "Nombre de bits à 1 : ",0
```

SECTION .bss

```
n         resd 1
compte1    resd 1
```

SECTION .text

```
global main
```

```
main:
```

```
    mov eax, prompt
    call print_string
    call read_int
    mov [n], eax
```

```
    mov dword [compte1], 0 ; compteur à 0
    mov ecx, 32            ; 32 bits à tester
    mov eax, [n]           ; EAX = nombre
```

```
boucle:
```

```
    shr eax, 1             ; Décale à droite : bit faible va dans CF
    jnc suivant            ; CF=0 -> pas de 1, on saute
```

```
    inc dword [compte1]    ; Si CF=1, on incrémente compte1
```

```
suitant:
```

```
    loop boucle            ; ECX--, continue tant que ECX>0
```

```
    mov eax, msg_result
    call print_string
    mov eax, [compte1]
    call print_int
    call print_nl
```

```
    mov ebx, 0
    mov eax, 1
    int 0x80
```

Explication pédagogique

- Loop : 32 passages — à chaque tour le registre EAX est décalé d'un bit vers la droite
- Carry Flag (CF) : après le décalage, il prend la valeur du bit qui "sort" (le bit le plus faible)
- Si CF=1 : c'était un bit à 1 → incrémentation du compteur
- Résultat en fin de boucle : nombre de bits à 1 dans la représentation binaire du nombre entré.

Exercice 9.

On cherche une autre manière de réaliser le programme de l'exercice précédent. En traitant manuellement quelques exemples, décrire ce que l'on obtient si l'on réalise le ET logique (instruction and) entre `eax` et `eax - 1`. En déduire un autre programme `E9.asm` calculant le nombre de bits valant 1 dans `eax`. Expliquer l'avantage de cette approche par rapport à celle de l'exercice précédent.

1. Qu'observe-t-on en faisant : `eax = eax & (eax - 1)` ?

Prenons des exemples manuels :

- Imagine `eax = 0b10100100` (soit `0xA4` en hex)
- `eax - 1 = 0b10100011`
- Donc : `10100100 AND 10100011 = 10100000`
- Remarque :
 - Cette opération met à zéro le bit à 1 le plus à droite de `eax` à chaque tour.
 - Autrement dit, à chaque itération, le nombre de bits à 1 diminue de 1.
- À chaque boucle, on fait :
 - `asm`
 - `eax = eax & (eax - 1)`et on incrémente un compteur.

Quand `eax` devient zéro, tous les bits à 1 ont été "enlevés" — donc notre compteur contient le nombre total de bits à 1.

2. Implémentation du programme `E9.asm`

Structure du programme :

- Lire un entier (enregistré dans une variable, par exemple `[n]`)
- Compteur de bits à 1 à zéro
- Tant que le nombre n'est pas zéro :
 - Appliquer `eax = eax & (eax - 1)`
 - Incrémenter le compteur
- Afficher le résultat

```
%include "asm_io.inc"
```



```

SECTION .data
prompt    db "Entrez un entier : ",0
msg_result db "Nombre de bits à 1 : ",0

SECTION .bss
n         resd 1
compte1   resd 1

SECTION .text
global main
main:
    mov eax, prompt
    call print_string
    call read_int
    mov [n], eax

    mov dword [compte1], 0 ; compteur à 0
    mov eax, [n]

boucle:
    cmp eax, 0
    je fin ; Si eax = 0, on termine

    inc dword [compte1] ; incrémente le compteur
    mov ebx, eax
    dec ebx ; ebx = eax - 1
    and eax, ebx ; eax = eax & (eax - 1)
    jmp boucle

fin:
    mov eax, msg_result
    call print_string
    mov eax, [compte1]
    call print_int
    call print_nl

    mov ebx, 0
    mov eax, 1
    int 0x80

```

3. Avantage de cette méthode par rapport à la précédente

- Efficace : Chaque tour retire un seul bit à 1, donc le nombre de boucles = nombre de bits à 1 (au lieu de 32 à chaque fois).
- Beaucoup plus rapide quand le nombre est peu "dense". Par exemple, pour un entier avec 2 bits à 1, la boucle ne tourne que 2 fois !
- Optimisé "par le bas" : travail proportionné au nombre réel de bits à 1, pas à la taille du registre.

Exercice 10.

Écrire un programme E10.asm qui lit en entrée une suite d'entiers compris entre 0 et 50 et terminée par -1 et affiche ensuite la liste dans l'ordre croissant des entiers entre 0 et 50 qui ne sont pas apparus dans la liste d'entrée. Par exemple, si la liste tapée en entrée est 4 1 15 1 2 -1,

Le programme affichera 0 3 5 6 7 8 9 10 11 12 13 14 16 17 18

1. Idée principale

- Créer un tableau de 51 octets, initialisé à 0, (pour se souvenir si l'entier i entre 0 et 50 a déjà été vu).
- Lis la suite d'entiers (entrée clavier), jusqu'à lire -1.
 - À chaque nombre lu (si dans), mets un 1 à sa position dans le tableau : $\text{tab}[n]=1$.
- Après la saisie, parcours de 0 à 50 : chaque i avec $\text{tab}[i]=0$ n'a pas été vu, on l'affiche.

```
%include "asm_io.inc"
```

```
SECTION .data
```

```
prompt    db "Entrez les entiers entre 0 et 50 (-1 pour terminer) : ",0
```

```
msg_result db "Entiers manquants : ",0
```

```
SECTION .bss
```

```
tableau    resb 51    ; 0 à 50 => 51 octets
```

```
n          resd 1      ; pour stocker l'entier lu
```

```
SECTION .text
```

```
global main
```

```
main:
```

```
    ; Afficher le prompt
```

```
    mov eax, prompt
```

```
    call print_string
```

```
lecture:
```

```
    call read_int
```

```
    mov [n], eax
```

```
    cmp eax, -1    ; -1 → fin de la lecture
```

```
    je fin_lecture
```

```
    cmp eax, 0
```

```
    jl lecture     ; nombre < 0 → ignorer
```

```
    cmp eax, 50
```

```
    jg lecture     ; nombre > 50 → ignorer
```

```
    mov bl, 1
```

```
mov [tableau + eax], bl ; marque le nombre comme présent
jmp lecture
```

fin_lecture:

```
; Affichage du résultat
mov eax, msg_result
call print_string
```

```
mov ecx, 0 ; compteur pour parcourir le tableau
```

affichage:

```
cmp ecx, 51
je fin_programme
```

```
mov al, [tableau + ecx]
cmp al, 0
jne skip_display ; si le nombre est marqué → passer
```

```
mov eax, ecx
call print_int
```

skip_display:

```
inc ecx
jmp affichage
```

fin_programme:

```
call print_nl
```

```
mov eax, 1 ; exit syscall
mov ebx, 0
int 0x80
```

Ce qu'il faut retenir pour l'oral/examen

- Utiliser un tableau de présence est plus efficace que trier ou scanner tout à chaque fois : c'est immédiat (un accès mémoire par entier).
- La condition `tab[i] == 0` repère ceux qui n'ont pas été vus.
- La valeur '1' dans `tab` signale que l'entier a été saisi au moins une fois.
- Après la saisie, parcourir de 0 à 50 et afficher les non-vus.

Exercice 11.

Écrire une nouvelle version E11.asm du programme de l'exercice précédent dans lequel on interdit toute lecture/écriture en mémoire. On n'utilise ainsi que les registres.

Astuce 3. Un registre est une suite de 32 bits qui peut être utilisée pour représenter l'absence ou la présence de tout entier compris entre 0 et 31. Par exemple, le fait que le bit d'indice 3 soit à 1 et le bit d'indice 9 soit à 0 dans `eax` signifie que `eax` représente un ensemble d'entiers contenant 3 mais pas 9.

1. Représentation en binaire

- Un registre comme eax a 32 bits.
- Si l'utilisateur saisit par exemple : 3 7 2 17 -1, tu mettras le bit 3, 7, 2, 17 à 1 dans eax.
- Pour poser un bit : utilise une opération OR avec un masque où le bon bit est à 1.

2. Saisie et mise à jour dans les registres

Quand tu lis une valeur entre 0 et 31 :

- Calcule : mov ebx, 1, puis shl ebx, valeur
- Fais : or eax, ebx
- Répète jusqu'à -1.

3. Affichage des non-vus

Pour i allant de 0 à 31 :

- Place le masque 1 << i dans ebx
- Fais : test eax, ebx
- Si résultat zéro : ce bit n'était pas vu ⇒ affiche i.
- Sinon, saute

```
%include "asm_io.inc"
```

```
SECTION .data
```

```
prompt_in  db "Entrez une suite d'entiers entre 0 et 50 terminee par -1 : ", 0
```

```
prompt_out db "Non vus : ", 0
```

```
SECTION .bss
```

```
seen_low   resd 1    ; bits de presence pour 0..31
```

```
seen_high  resd 1    ; bits de presence pour 32..50
```

```
SECTION .text
```

```
global main
```

```
main:
```

```
    ; Initialiser les bitsets à 0
```

```
    mov dword [seen_low], 0
```

```
    mov dword [seen_high], 0
```

```
    ; Afficher le prompt
```

```
    mov eax, prompt_in
```

```
    call print_string
```

```
; ===== LECTURE DES ENTIER =====
```

```
lire_nb:
```

```
    call read_int    ; EAX = entier lu
```

```
    cmp eax, -1
```

```
    je fin_lecture    ; -1 -> fin de la saisie
```

```
    cmp eax, 0
```

```
    jl lire_nb        ; < 0 (≠ -1) -> ignoré, on relit
```

```
cmp eax, 50
jg lire_nb      ; > 50 -> ignoré, on relit
```

```
; Ici: 0 <= EAX <= 50
```

```
mov ebx, eax    ; EBX = n
cmp ebx, 32
jl set_low      ; si n < 32 -> seen_low
```

```
; ----- cas n entre 32 et 50 -> seen_high -----
sub ebx, 32     ; EBX = position (0..18)
mov ecx, ebx    ; ECX = position du bit
mov eax, 1
shl eax, cl     ; EAX = 1 << position
or [seen_high], eax ; mettre ce bit à 1
jmp lire_nb
```

```
set_low:
; ----- cas n entre 0 et 31 -> seen_low -----
mov ecx, ebx    ; ECX = n
mov eax, 1
shl eax, cl     ; EAX = 1 << n
or [seen_low], eax ; mettre ce bit à 1
jmp lire_nb
```

```
; ===== FIN DE LA LECTURE =====
```

```
fin_lecture:
mov eax, prompt_out
call print_string
```

```
mov ecx, 0      ; ECX = i = 0
```

```
boucle_i:
cmp ecx, 51     ; tant que i < 51
jge fin_programme
```

```
mov eax, ecx    ; EAX = i
cmp eax, 32
jl test_low     ; si i < 32 -> seen_low
```

```
; ----- test i dans seen_high -----
sub eax, 32     ; position = i - 32 (0..18)
mov edx, eax    ; EDX = position
mov eax, 1
mov cl, dl      ; CL = position
shl eax, cl     ; masque = 1 << position
mov ebx, [seen_high]
```

```
test ebx, eax      ; test du bit correspondant
jnz pas_manquant  ; si bit = 1 -> i déjà vu
```

```
; bit = 0 -> i NON vu, on l'affiche
mov eax, ecx
call print_int
call print_espace
jmp suite_i
```

```
test_low:
; ----- test i dans seen_low -----
mov edx, eax      ; EDX = i
mov eax, 1
mov cl, dl        ; CL = i
shl eax, cl       ; masque = 1 << i
mov ebx, [seen_low]
test ebx, eax
jnz pas_manquant  ; si bit = 1 -> i déjà vu
```

```
; bit = 0 -> i NON vu
mov eax, ecx
call print_int
call print_espace
```

```
suite_i:
inc ecx          ; i++
jmp boucle_i
```

```
pas_manquant:
inc ecx
jmp boucle_i
```

```
fin_programme:
call print_nl
mov ebx, 0
mov eax, 1
int 0x80
```

%include "asm_io.inc" => si on avait besoin que de 32b plus simple

SECTION .data

prompt db "Entrez un entier entre 0 et 31 (-1 pour finir) : ", 0

prompt_out db "Non vus : ", 0

SECTION .bss

n resd 1

i resd 1

SECTION .text

global main

main:

```

        xor eax, eax            ; registre presence, tous bits = 0

; --- lecture de la liste ---
lire_nb:
    mov ebx, prompt
    call print_string
    call read_int
    mov [n], eax

    cmp eax, -1
    je touslus

    cmp eax, 0
    jl lire_nb
    cmp eax, 31
    jg lire_nb

    mov ecx, 1
    mov ebx, [n]
    shl ecx, cl                ; masque = 1 << valeur entrée
    or eax, ecx                ; place le bit à 1 dans EAX

    jmp lire_nb

; --- affichage des non vus ---
touslus:
    mov ebx, prompt_out
    call print_string

    mov dword [i], 0

boucle_aff:
    mov ecx, [i]
    cmp ecx, 32
    jge fin

    mov edx, 1
    shl edx, cl
    test eax, edx              ; Si ce bit est à 0, non vu
    jnz suite

    mov eax, [i]
    call print_int
    call print_espace

suite:
    inc dword [i]
    jmp boucle_aff

```

fin:

```
call print_nl  
mov ebx, 0  
mov eax, 1  
int 0x80
```

Explication pédagogique

- Représentation : enregistres, chaque bit représente la présence (1) ou non (0) d'un entier.
- Ajout d'un entier : via l'opération OR et le décalage du masque.
- Recherche des absents : via test (AND binaire) suivi d'un saut conditionnel.

Avantages

- Très rapide : l'accès à un "bit" d'état est instantané (pas de boucle/mémoire, juste un décalage et une opération logique).
- Zéro mémoire utilisée (hors les deux registres techniques pour n et i lors de la boucle).
- En pratique, largement plus rapide que la méthode "tableau", surtout si la liste d'entrée est longue.